Air University

Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: April 30, 2026

Lab # 11

1. Code

```
#include "CImg.h"
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
using namespace cimg_library;

//  Steganography – LSB Embed & Extract
//  Cover image : cover.bmp  (736 x 397 RGB)
//  Capacity    : 109,572 characters
class Steganography
{

    string text;
    CImg<unsigned char> img;

public:
    // Load image from file
    Steganography(string imgFile)
    {
        img.load(imgFile.c_str());
        cout << "Image loaded  : " << imgFile << "\n";
        cout << "Dimensions    : " << img.width()
             << " x " << img.height()
             << " x " << img.spectrum() << " channels\n";
        cout << "Max capacity  : "
             << (img.width() * img.height() * img.spectrum()) / 8
             << " characters\n";
    }

    // Read secret message from secret.txt
    void readSecret()
    {
        ifstream f("secret.txt");
        if (!f.is_open())
        {
            cout << "Error: secret.txt not found!\n";
            return;
        }
        getline(f, text);
        f.close();
        cout << "\nSecret message : \"" << text << "\"\n";
        cout << "Length         : " << text.length() << " characters\n";
        cout << "Bits needed    : " << text.length() * 8 << " bits\n";
    }

    // Return specific bit of a character
    // charIdx = which character, bitPos = which bit (0=LSB to 7=MSB)
```

```
int getBit(int charIdx, int bitPos)
{
    return (text[charIdx] >> bitPos) & 1;
}

// Set the LSB of a pixel byte to our secret bit
// (pixelVal & ~1) clears bit 0, then OR sets it to our bit
unsigned char setLSB(unsigned char pixelVal, int bit)
{
    return (pixelVal & ~1) | bit;
}

// EMBED: hide secret message bits into image LSBs
void embed()
{
    int totalChars = text.length();
    int totalBitsNeeded = totalChars * 8;
    int capacity = img.width() * img.height() * img.spectrum();

    cout << "\n--- Embedding ---\n";
    cout << "Image capacity : " << capacity << " bits\n";
    cout << "Bits needed    : " << totalBitsNeeded << " bits\n";

    if (totalBitsNeeded > capacity)
    {
        cout << "Error: Image too small for this message!\n";
        return;
    }

    int bitPos = 0;
    int charIdx = 0;

    for (int x = 0; x < img.width(); x++)
    {
        for (int y = 0; y < img.height(); y++)
        {
            for (int c = 0; c < img.spectrum(); c++)
            {
                if (charIdx < totalChars)
                {
                    int secretBit = getBit(charIdx, bitPos);
                    unsigned char original = img(x, y, 0, c);
                    unsigned char modified = setLSB(original, secretBit);
                    img(x, y, 0, c) = modified;
                    bitPos++;
                    if (bitPos == 8)
                    {
                        bitPos = 0;
                        charIdx++;
                    }
                }
            }
```

```
        }
    }

    cout << "Characters hidden : " << totalChars << "\n";
    cout << "Bits used         : " << totalBitsNeeded << "\n";
}

// Save the stego image
void saveImage()
{
    img.save("stego.bmp");
    cout << "Stego image saved : stego.bmp\n";
}

// EXTRACT: recover secret message from stego image
// Reads LSB of each pixel channel in same order as embedding
// Assembles bits into characters 8 at a time
string extract(string stegoFile, int knownLength)
{
    CImg<unsigned char> stego;
    stego.load(stegoFile.c_str());

    string recovered = "";
    int currentByte = 0;
    int bitCount = 0;
    int charsRead = 0;

    for (int x = 0; x < stego.width(); x++)
    {
        for (int y = 0; y < stego.height(); y++)
        {
            for (int c = 0; c < stego.spectrum(); c++)
            {
                if (charsRead < knownLength)
                {
                    int lsb = stego(x, y, 0, c) & 1;
                    currentByte |= (lsb << bitCount);
                    bitCount++;
                    if (bitCount == 8)
                    {
                        recovered += (char)currentByte;
                        currentByte = 0;
                        bitCount = 0;
                        charsRead++;
                    }
                }
            }
        }
    }
    return recovered;
}
```

```
    // Run full embed workflow
    void runEmbed()
    {
        readSecret();
        embed();
        saveImage();
    }
};

//  MAIN — Menu
int main()
{
    int choice;

    do
    {
        cout << "\n===== LSB Steganography Program =====\n";
        cout << "Cover image : cover.bmp (736 x 397 RGB)\n";
        cout << "-------------------------------------\n";
        cout << "1. Embed secret message\n";
        cout << "2. Extract secret message\n";
        cout << "3. Exit\n";
        cout << "Enter choice (1-3): ";
        cin >> choice;

        if (choice == 1)
        {

            Steganography s("cover.bmp");
            s.runEmbed();
        }
        else if (choice == 2)
        {

            int len;
            cout << "\nEnter number of characters to extract: ";
            cin >> len;

            cout << "\n--- Extraction ---\n";
            Steganography s("stego.bmp");
            string msg = s.extract("stego.bmp", len);

            cout << "Extracted message : \"" << msg << "\"\n";

            ofstream out("extracted.txt");
            out << msg;
            out.close();
            cout << "Saved to extracted.txt\n";
        }
        else if (choice == 3)
        {
            cout << "Goodbye!\n";
```

```
        }
        else
        {
            cout << "Invalid choice! Enter 1, 2, or 3.\n";
        }

    } while (choice != 3);

    return 0;
}
```

Explanation:

1. **Purpose and Library:** This program implements LSB (Least Significant Bit) Steganography, a method for hiding a secret text message within an image file (cover.bmp) by making subtle adjustments to pixel values.
2. **Image Loading and Capacity:** The Steganography class opens the image and calculates its maximum capacity for storing data by multiplying the image's width, height, and the number of color channels. This total is then divided by 8 because each character requires 8 bits.
3. **Secret Message Input:** The `readSecret()` function retrieves a hidden message from a file named `secret.txt` and calculates the total bits needed to embed that message.
4. **Bit Manipulation Logic:** The `getBit()` helper function extracts individual bits from a character, while `setLSB()` uses a bitwise AND operation (& ~1) to clear the least significant bit of a pixel value, followed by a bitwise OR operation to insert the secret bit.
5. **Embedding Process:** The `embed()` function traverses the image's pixels and color channels (R, G, B), replacing the least significant bit of each byte with a bit from the secret message until the entire message is securely embedded.
6. **Data Preservation:** After the message has been embedded, the `saveImage()` function writes the modified pixel data to a new file called `stego.bmp`, leaving the original `cover.bmp` intact.
7. **Extraction Process:** The `extract()` function undoes the embedding process by reading the least significant bits from each pixel in the `stego.bmp` file and putting those bits back together into 8-bit characters to retrieve the original message.
8. **User Interface:** The `main()` function offers a command-line menu that allows users to select between embedding a message, extracting one, or exiting the program.
9. **Length Dependency:** During extraction, the user must provide the known length of the hidden message because the program does not include a "stop" marker within the image.
10. **Output Generation:** After successfully extracting the message, the program displays it in the terminal and saves the recovered text to a file named `extracted.txt`.

2. Compilation & Output

Using ctrl+shift+B

```
*  Executing task: g++ -std=c++17 -DCIMG_DISPLAY=0 -Dcimg_display=0 -o 'D:\University\Cryptography\Lab\Lab 11 (St
sity\Cryptography\Lab\Lab 11 (Stegnography)\stego.cpp'

*  Terminal will be reused by tasks, press any key to close it.
```

Using ./stego.cpp

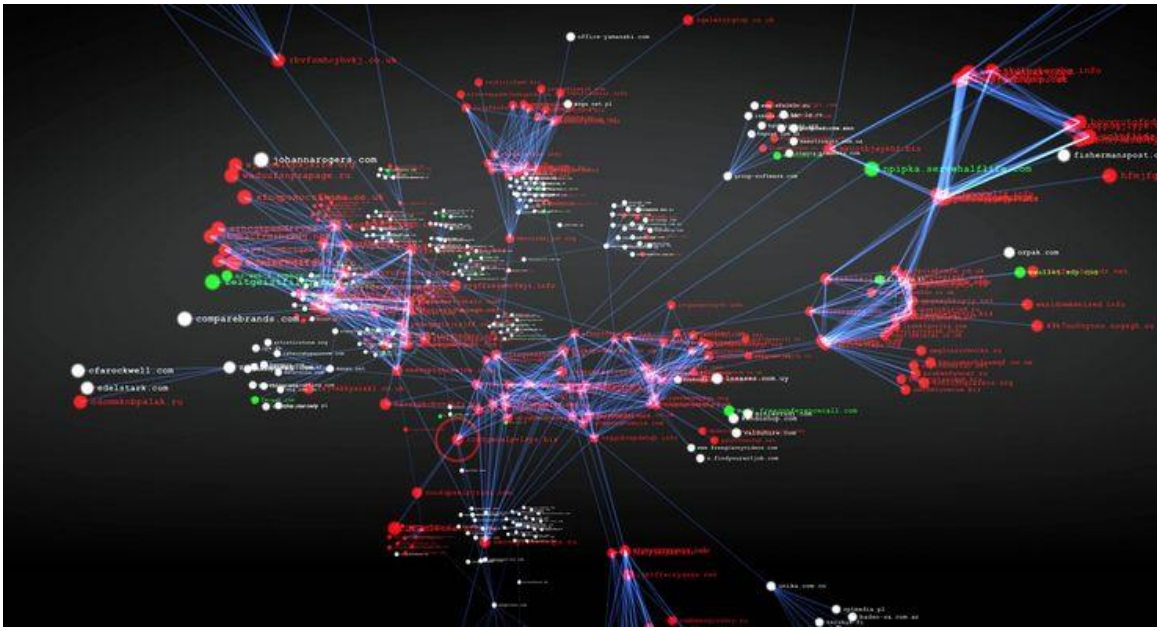
```
PS D:\University\Cryptography\Lab\Lab 11 (Stegnography)> ./stego.exe
```

Output

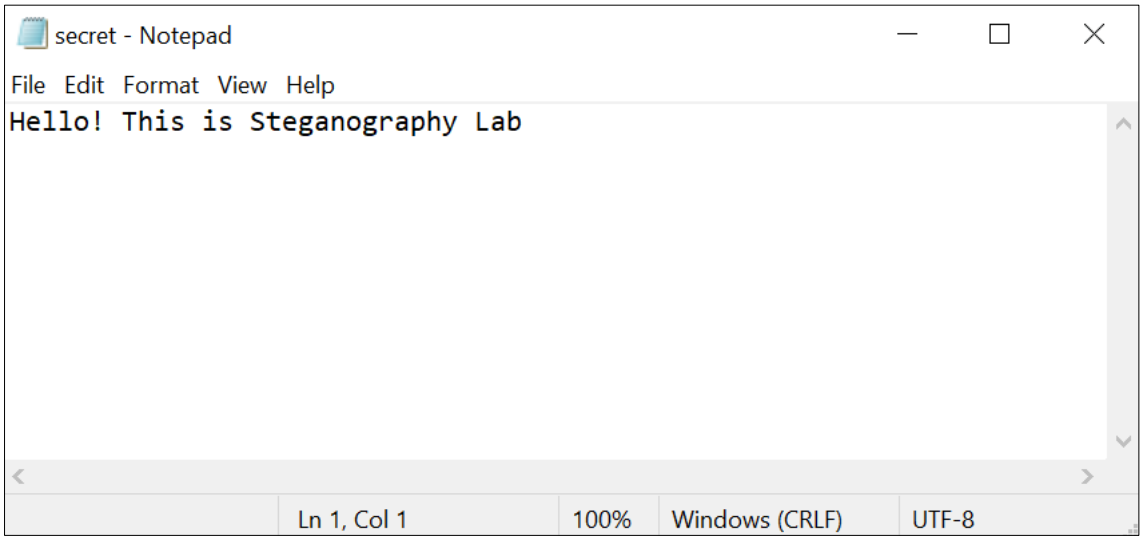
```
===== LSB Steganography Program =====
Cover image : cover.bmp (736 x 397 RGB)
-------------------------------------
1. Embed secret message
2. Extract secret message
3. Exit
Enter choice (1-3): 1
Image loaded  : cover.bmp
Dimensions    : 736 x 397 x 3 channels
Max capacity  : 109572 characters

Secret message : "Hello! This is Steganography Lab"
Length         : 32 characters
Bits needed    : 256 bits

--- Embedding ---
Image capacity : 876576 bits
Bits needed    : 256 bits
Characters hidden : 32
Bits used         : 256
Stego image saved : stego.bmp
```

```
===== LSB Steganography Program =====
Cover image : cover.bmp (736 x 397 RGB)
-------------------------------------
1. Embed secret message
2. Extract secret message
3. Exit
Enter choice (1-3): 2

Enter number of characters to extract: 32

--- Extraction ---
Image loaded  : stego.bmp
Dimensions    : 736 x 397 x 3 channels
Max capacity  : 109572 characters
Extracted message : "Hello! This is Steganography Lab"
Saved to extracted.txt

===== LSB Steganography Program =====
Cover image : cover.bmp (736 x 397 RGB)
-------------------------------------
1. Embed secret message
2. Extract secret message
3. Exit
Enter choice (1-3): 3
Goodbye!
```

3. Image Used



4. Secret message



5. Resource utilized

CImg	29-Apr-26 7:10 PM	C Header Source File	3,411 KB
cover	30-Apr-26 9:43 AM	BMP File	857 KB
download	30-Apr-26 9:36 AM	BMP File	857 KB
extracted	30-Apr-26 10:10 AM	Text Document	1 KB
secret	30-Apr-26 9:55 AM	Text Document	1 KB
stego	30-Apr-26 10:09 AM	BMP File	857 KB
stego	30-Apr-26 10:13 AM	C++ Source File	7 KB
stego	30-Apr-26 10:05 AM	Application	1,330 KB

- End of Lab Task -